Ders 9 Genişletilmiş Notu: Yığın, İnfix-Postfix Dönüşümü ve Kuyruk

Ders 9 Genişletilmiş Notu: Yığın, İnfix-Postfix Dönüşümü ve Kuyruk

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Döngü Kullanımında Sık Yapılan Hatalar
- 2. Yığın Nedir? Günlük Kullanımdan Örnekler
- 3. Yığının Mantığı: Soyut Veri Tipi, `push`, `pop` ve `top`
- 4. Yığını Dizi ile Yazmak
- 5. İnfix İfadeyi Postfix İfadeye Çevirme
- 6. Postfix İfadeyi Yığınla Hesaplama
- 7. Derleyicinin İki Yığınla Hesaplaması
- 8. Parantez Denetimi
- 9. Geri Alma ve Yineleme: İki Yığın
- 10. Kuyruk (Queue)

1. Döngü Kullanımında Sık Yapılan Hatalar

Hoca derse, yeni konuya geçmeden önce okuduğu quizlerde gördüğü iki döngü hatasını anlatarak başladı. Bu hatalar quizdeki soruya özgü değildir, her programda karşımıza çıkabilir. Bu yüzden genel bir örnek üzerinden gösterildi: n elemanlı bir `a` dizisinde bir `x` değeri aranıyor. Ortak nokta şudur: Yapılan iş doğru sonuç verse bile, kodun verimli (efektif) yazılması da bir ölçüttür.

1.1 Bulunduğu Hâlde Döngüden Çıkmamak

Hocanın en çok dikkatini çeken hata, adım sayısı belli olmayan bir aramada `for` kullanmak ve aranan bulunduğu hâlde döngüden çıkmamaktı. Böyle bir kod `a[i]`, `x`'e eşit olunca `bul = 1` yapıyor, ama döngü dizinin sonuna kadar dönmeye devam ediyor. İş ancak döngü bittikten sonra `bul` değerine bakılarak yapılıyor:

```
for (i = 0; i < n; i++)
    if (a[i] == x)
        bul = 1;        /* bulundu ama döngü sürüyor */

if (bul == 1)
{
    /* işlemi yap */
}
```

Burada yapılması gereken, `while` kullanmak ve `a[i]`, `x`'e eşit olduğunda döngüden çıkmaktır. Hocanın gerekçesi açıktır: Bir milyon elemanınız varsa ve aradığınızı birinci adımda bulduysanız, içeride bir milyon kez gezmenin bir anlamı yoktur.

> **Hocanın vurgusu:** İkinci sınıf ikinci dönemde artık adım sayısı belli olan ve olmayan döngülerde ne yapacağınızı biliyor olmalısınız. Adım sayısı belli değilse `while` kullanın, iş bitince çıkın.

1.2 `while` Döngüsünde Dizi Sınırını Unutmak

İkinci hata da `while` ile ilgilidir. Bu kez `while` kullanılıyor, ama koşulda yalnızca aranan değere bakılıyor:

```
while (a[i] != x)
{
    /* işi yap */
    i++;
}
```

Sorun şudur: Ya o şart hiç sağlanmazsa? Quizdeki renk sorusunda da tam bu durum vardı. Aranan renk daha önce hiç kullanılmamış olabilir, yani dizide olmayabilir. Böyle bir kodun o durumu yakalama şansı yoktur. Kaç eleman olduğu biliniyorsa önce dizinin sonuna gelinip gelinmediği, sonra aranan koşul kontrol edilmelidir:

```
while (i < n && a[i] != x)
    i++;
```

Hoca bunların sınav heyecanıyla ilgisi olmadığını, artık yerleşmiş olması gereken alışkanlıklar olduğunu söyledi. Dizi üzerinde dönen bir döngüde elemanın hiç olmama durumu da düşünülmeli, "dizinin sonuna gelince çık" denmelidir.

> **Ek bilgi (derste anlatılmadı):** Koşuldaki sıra önemlidir, çünkü C'de `&&` soldan sağa değerlendirilir ve sol taraf yanlışsa sağ taraf hiç hesaplanmaz. `i < n` önde olunca `i == n` iken `a[n]` okunmaz. Üç sürüm, aranan değerin dizide olmadığı bir girdiyle `gcc -Wall -fsanitize=address` altında çalıştırıldı: `while (a[i] != x)` ve ters sıralı `while (a[i] != x && i < n)` dizinin dışını okuyarak hata verdi, `while (i < n && a[i] != x)` hatasız bitti.

2. Yığın Nedir? Günlük Kullanımdan Örnekler

Hoca bu dersin konusunu, bilgisayarı kullanırken farkına varmadan belki de en çok kullandığımız veri yapısı olarak tanıttı: yığın (stack). Önce tanım vermek yerine dört örnek gösterdi, sonra bu örneklerdeki ortak noktayı çıkardı.

2.1 Web Üzerinde Gezinmek

Bir gazetenin sitesine bağlandığınızı düşünün. Oradan yazarlar sayfasına, oradan da bir yazarın sayfasına geçtiniz. Sol menüyü kullanmadığımızı varsayalım. Geri tuşuna basınca nereye dönersiniz? Yazarlar sayfasına. Bir daha basınca gazeteye dönersiniz. Yazarın sayfasından doğrudan gazeteye dönmezsiniz. Bulunduğunuz noktadan her seferinde yalnızca bir öncekine gidersiniz.

2.2 Metin Düzenleyicide Geri Alma

Hoca kalemiyle ekrana "Ali" yazdı ve geri alma (undo) işlemini gösterdi. Silme, yazılan sıranın tersine ilerledi: önce `i`'nin noktası, sonra `i`, sonra `l`, sonra `A`'nın çizgisi, en son `A`'nın kendisi. Yani `A` önce silinmiyor. Geri alma her zaman en son yapılan işlemi alır, sonra ondan bir öncekini alır ve böylece sondan başa doğru gider.

2.3 Fonksiyon Çağrıları

Üçüncü örnekte bir `main` fonksiyonu `F1`'i, `F1` de `F2`'yi çağırıyor. `F2` bitince `main`'e dönülmez, `F2`'yi çağıran yere yani `F1`'e dönülür. `F1` bitince de `F1`'i çağıran yere, yani `main`'e dönülür. Burada da bir hiyerarşi vardır ve sıra bozulmaz: En son `F2` bittiği için önce ondan bir önceki duruma dönülür. Hoca bunu undo ile aynı mantık olarak gösterdi: "Pat diye `main`'e geri dönmek yok."

Özyineleme (recursion) de aynı ilkeyle çalışır. Fonksiyonlar böyle çağrıldığında sistem yığın kullanarak bulunduğu noktadan adım adım geri döner.

2.4 Veri Yapıları İhtiyaçtan Doğar

Hoca burada ilk derste konuşulan bir noktayı hatırlattı: Veri yapıları, belli algoritmaların ihtiyaçlarına göre oluşturulmuş yapılardır.

- **Dizi:** İstediğiniz göze doğrudan erişirsiniz. Beşinci gözü istiyorsanız beşinci göze, dokuzuncu gözü istiyorsanız dokuzuncu göze gidersiniz.
- **Bağlı liste (linked list):** Herkes yalnızca kendinden sonra geleni gösterir. Bunun amacı, bellekte ayrı yerlerde duran bilgilerin birbirini kaybetmemesidir. Böylece ardışık duran bir diziye erişilmiş gibi arka arkaya bilgilere erişilebilir.
- **Yığın:** Son giren ilk çıkar (LIFO, last in first out). Son gelenle ilk işi yapmam gereken bir algoritmaya ihtiyacım varsa kullanacağım veri yapısı yığındır.

Verilen üç örneğin ortak noktası budur. Aradan bir elemanla iş yapılmak istenmez. Her adımda en son bulunulan noktayla iş yapılır ve başa ya da aradaki bir yere değil, bir önceki duruma dönülür. Yığın, bu hâliyle doğrusal (lineer) bir veri yapısıdır.

2.5 Aritmetik İfadeler

Dördüncü kullanım alanı matematik ifadelerdir. `3 + 4 * 5` yazdığımızda derleyici parantez olmamasına rağmen önce 4×5 işlemini yapar, sonra sonucu 3 ile toplar. Fakat derleyici ifadeyi soldan okumaya başlar. Toplamaya, çarpmadan önce gelir. O hâlde toplamayı yapmadan çarpmayı yapıp sonra toplamaya geri dönmesi, yani işlemleri kaybetmemesi gerekir. Bu da yığınla sağlanır. Bunun ayrıntısı dersin ilerleyen bölümlerinde anlatıldı.

3. Yığının Mantığı: Soyut Veri Tipi, `push`, `pop` ve `top`

3.1 Soyut Bir Veri Tipi

Yığın yapısı donanımda da vardır. Hoca bunun Assembly dersinde ve programlama derslerinde anlatılmış olduğunu tahmin ettiğini söyledi. Fakat burada kullandığımız, soyut (abstract) bir veri tipidir. Soyuttan kasıt şudur: Gerçekte öyle ayrı bir yapı yoktur. Normal bir dizi kullanırız, ama dizinin istediğimiz elemanına erişmeyiz. Diziyi yalnızca yığın mantığının izin verdiği şekilde kullanırız.

Hoca diziyi dik çizerek gösterdi: En alttaki göz 0. göz, en üstteki göz $n-1$. gözdür. A gelince 0. göze, B gelince 1. göze, C gelince 2. göze yerleşir. En üstteki eleman alınmak istenince C dışarı çıkar ve yığında A ile B kalır.

3.2 İki Temel İşlem ve `top`

Yığın işlemleri için iki temel fonksiyon vardır:

- `push`: Gelen elemanı yığına yerleştirir.
- `pop`: Yığının en üstündeki bilgiyi dışarı alır.

Bir de nerede olduğumuzu bilmemiz gerekir. Bunu `top` değişkeni tutar. `top`, yığının en üstünde bir sonraki elemanın yerleşeceği gözü gösterir. Yığın boşken `top` 0. gözü gösterir. Hocanın adım adım örneği şöyledir:

İşlem	Yapılan	Dizi (0. gözden yukarı)	`top`	`x`
Başlangıç		boş	0	
`push A`	A, 0. göze yerleşir, `top` artar	A	1	
`push B`	B, 1. göze yerleşir, `top` artar	A B	2	
`pop`	`top` azalır, o gözdeki değer `x`'e alınır	A	1	B
`push C`	C, `top`'un gösterdiği 1. göze yerleşir	A C	2	B

> **Ders sonrası not:** Hoca `top` için "pointer" ve "adres tutan değişken" dedi. Dersin kodunda `top` bir gösterici değil, `int` türünde bir dizi indisidir. Kastedilen, dizide nerede olunduğunu göstermesidir.

3.3 Yığının Bir Sınırı Vardır: Stack Overflow

Yığın bir dizi olduğuna göre en büyük bir eleman sayısı vardır. Diziyi ister dinamik olarak ayırın ister sabit uzunluklu tanımlayın, sonuçta belli bir alanı tutar ve onu kullanırsınız.

Hoca bunun gündelik programlamadaki karşılığını sordu. Özyinelemeli (recursive) bir fonksiyonu düzgün yazamadıysanız ve fonksiyon durmadan kendini çağırıyorsa sistem "stack overflow" hatası verir. Nedeni budur: Fonksiyon çağrıları için yığın kullanılır, bitmeyen çağrılar yığını doldurur ve taşma olur. Bir öğrenci de bu hatayı özyinelemeli fonksiyonlarda aldığını söyledi.

Sistem bu durumda "yığın doldu, biraz daha yer açayım" demez. Çünkü böyle bir şey istenmez ve buna gerek de yoktur. Yığın dolduysa zaten bir sıkıntı vardır. Bu tür uygulamalarda ne kadar yer gerektiği bellidir. Hocanın web örneğine dönüşü de aynı yöndedir: Arka arkaya on bin sayfa açmaya da sistem izin vermez, onun da bir sınırı vardır.

4. Yığını Dizi ile Yazmak

4.1 Yapı ve İlk Değer Verme

Yığın için bir veri yapısı (struct) gerekir. İçinde iki ana bilgi olur. Birincisi yığın işlemlerinin yapılacağı dizi, ikincisi yığında nerede olunduğunu, yani gelen elemanın nereye yerleşeceğini ya da çıkan elemanın nereden çıkacağını gösteren `top`. En az bunlar yeterlidir.

Yapı ve yardımcı fonksiyonlar (ekrandan, 00:40:30; kısa nottan):

```
#define MAXSIZE 100
typedef struct
{
    int items[MAXSIZE];
    int top;
} STACK;

void initialize(STACK *s)
{
    s->top = 0;
}

int empty(STACK *s)
{
    if (s->top == 0)
        return 1;
    else
        return 0;
}

int full(STACK *s)
{
    if (s->top == MAXSIZE)
        return 1;
    else
        return 0;
}
```

İlk değer verme (initialize), yığının göstericisinin başlangıçta 0. gözü göstereceğini söylemek demektir. Bunu `main` içinde `top`'u sıfırlayarak da yapabilirsiniz. Bu da çalışan bir çözümdür. Fakat hoca bunu ayrı bir fonksiyona koymayı önerdi.

> **Hocanın vurgusu:** `initialize`, `display`, bilgileri alan ve yazan fonksiyonlar ayrı ayrı olsun. Tasarımınızı anlamaya çalışan kişi işin nerede yapıldığını bakar bakmaz görür. Yapılan işlerin takibi çok daha ferah olur.

4.2 Boş ve Dolu Kontrolü

Hoca öğrencilere `push` ve `pop` yazılırken neye dikkat edilmesi gerektiğini sordu. Cevap iki kontrol oldu:

- `push` ederken yığının dolu olup olmadığı kontrol edilir. Doluysa eleman yerleştirilmez.
- `pop` ederken yığının boş olup olmadığı kontrol edilir. `top` 0 ise yığında hiç eleman yoktur, `pop` yapılamaz.

Yanlış kullanım her zaman olabilir. Birisi boş yığından eleman çekmeye kalkabilir. Bu yüzden kontrol gereklidir.

`empty` yazmak kolaydır. Başlangıçta `top` 0 yapıldığına göre `top` 0 iken yığın boştur. Fonksiyon boşsa 1, içinde eleman varsa 0 döndürür.

`full` için hoca, yığının ne zaman dolu olduğunu sordu.

Soru: `top`, `MAXSIZE - 1`'e eşitse yığın dolu olmaz mı?

Cevap: Hayır. Eşitse hâlâ o göze yerleşebilir. Elemanlar 0. gözden başlayıp yukarı doğru yerleşir ve son eleman `MAXSIZE - 1`. göze yerleşir. O anda `top`, `MAXSIZE` gözünü gösterir. Burası artık yığına ait değildir. Yerleştirmek istenirse "orası sana ait değil" demesi gerekir. Yani `top == MAXSIZE` ise yığın doludur.

Hoca ekranda bir yerde `MAX`, bir yerde `MAXSIZE` yazdığını fark edip belirtti. Bu notta her yerde `MAXSIZE` kullanılmıştır.

Bu yapıda bağlı liste kullanılmadığını hoca bir kez daha söyledi. Elemanlar dizi mantığında arka arkaya saklanır. Baştan dinamik ayırsanız bile belli bir yeriniz vardır. "Doluysa `realloc` et, daha geniş yer tut" gibi kullanımlar görülür, ama genelde yapılmaz. Mantık, yığın bölgesinin yerinin belli olması ve orasının kullanılmasıdır.

4.3 Neden Adres Gönderiyoruz?

Hoca dikkat çekici bir soru sordu: `empty` ve `full` yığını değiştirmiyor. `initialize` içinde `top` değişiyordu, ama burada hiçbir şey değişmiyor. Yine de neden yapının kendisi değil adresi gönderiliyor? Yapıyı değer olarak göndermek de hata olmazdı.

Soru: Değişmediği hâlde neden `s` değil de adresi gönderiliyor?

Cevap (bir öğrencinin önerisiyle): Adres olarak gönderilmezse yapı kopyalanır. Buna ihtiyaç yoktur. Adres, yani yalnızca referans gönderilince performans açısından daha iyi olur. Hoca doğru cevabın bu olduğunu söyledi ve açıkladı: Struct dediğinizde aslında bir sürü bilgiyi bir arada tutmak istiyorsunuzdur. Değer olarak gönderirseniz içindeki bütün bilgiler fonksiyon tarafına kopyalanır. Adres gönderirseniz fonksiyon, bilginin bellekte durduğu yer üzerinde iş yapar.

> **Hocanın vurgusu:** Yapıyı değer olarak değil adres olarak göndermek verimlidir. Sistem boşu boşuna bilgileri kopyalamakla vakit kaybetmez. Bunu Yapısal Programlama dersinde görmüş olmanız gerekir.
>
> **Ek bilgi (derste anlatılmadı):** Hoca, dizilerin normalde kopyalanmayıp başlangıç adresiyle gönderildiğini de belirtti. Bu, doğrudan dizi parametresi için geçerlidir. Dizi bir struct'ın içindeyse ve struct değer olarak gönderilirse dizinin tamamı da kopyalanır. `STACK` yapısıyla `gcc -Wall` altında denendi: Değer alan fonksiyonun `items[0]` ve `top` üzerinde yaptığı değişiklik çağırana yansımadı, adres alan fonksiyonunki yansıdı.

4.4 `push` Fonksiyonu

`push` için iki parametre gerekir: yerleştirilecek değer `x` ve yerleştirilecek yığın. Aslında `push` sonunda döndürülecek bir şey yoktur. Ama işlemin yapılıp yapılamadığını döndürmek yararlı

olabilir. Bu yüzden dönüş tipi `int` seçildi. Fonksiyonlar kısa olduğu için hoca akış diyagramı çizmek yerine doğrudan kod yazdı.

Dolu değilse eleman `top`'un gösterdiği göze yazılır, sonra `top` bir artırılır. Çünkü `top` A yerleştirilirken 0'ı gösteriyordu ve yerleştikten sonra 1'i göstermelidir.

`push` ve `pop` (ekrandan, 01:05:10; kısa nottan):

```
int push(int x, STACK *s)
{
    if (full(s))
        return 0;           /* yığın dolu, push yapılamadı */
    else
    {
        s->items[s->top] = x;
        s->top++;
        return 1;           /* push başarılı */
    }
}

int pop(STACK *s, int *x)
{
    if (empty(s))
        return 0;           /* yığın boş, pop yapılamadı */
    else
    {
        --s->top;
        *x = s->items[s->top];
        return 1;           /* pop başarılı */
    }
}
```

Hoca `top`'u artırmayı dikkat çeksin diye ayrı satırda yazdı. Atama sırasında `s->items[s->top++] = x;` da yazılabilir.

> **Ders sonrası not:** 4.1 ve 4.4'teki fonksiyonlar, 3.2'deki `push A`, `push B`, `pop`, `push C` izini ve doluyken `push` durumunu deneyen küçük bir `main` ile `gcc -Wall` altında uyarısız derlenip çalıştırıldı. `pop` sonrası `x` B, `top` 1 oldu. C, 1. göze yerleşti. Boş yığında `pop` 0, `MAXSIZE` eleman sonrası `push` 0 döndürdü. `main` derste yazılmadığı için buraya alınmadı.

4.5 `return` Olan `if`'ten Sonra Neden `else`?

Hoca kodda `return 0`'dan sonra `else` yazdığını gösterip neden yazdığını sordu. Yığın doluysa `return` ile fonksiyondan zaten çıkılır. Bu `else` yazılmasa da program doğru çalışır. Bir öğrenci kendisinin `else` yazmayacağını söyledi.

Hocanın cevabı anlaşılırlıkla ilgilidir. Bir proje grubunda çalıştığınızı, kodunuzu sonradan başka birinin güncellediğini düşünün. Kod `else` olmadan şöyle olsun:

```
if (full(s))
    return 0;
s->items[s->top] = x;
s->top++;
return 1;
```

Kodu sonradan değiştiren kişi `return 0`'ı kaldırıp yerine `x = x + bir_sey;` gibi başka bir şey yazarsa ne olur? `else` olmadığı için şart sağlansa da akış dümdüz aşağı iner. Yığın doluyken 4.2'de "yığına ait değil" denen göze yazılır. Hoca bunun sık rastlanan bir sorun olduğunu söyledi. Bu kod çok açık olduğu için belki böyle bir hata olmaz. Ama başka bir yerde `if (a == x) return x;` yazıp `else` koymadan devam ederseniz, sonradan gelen kişi `return x` yerine başka bir iş yazar, `else` olmadığı için uyanamaz ve program yanlış çalışır.

> **Hocanın vurgusu:** `return` ile fonksiyondan çıkıldığını hepimiz biliyoruz. Yine de uzun zaman sonra unutularak yapılabilecek hataları engellemek için `else`'in nerede olduğunu açıkça gösterin. Kodun doğruluğunu garantiye almak için bu tür anlaşılırlık işlerine dikkat edin.

4.6 `pop` Fonksiyonu

`pop`'ta önce yığının boş olup olmadığına bakılır. Boşsa iş yapılamaz ve `return` edilir. Boş değilse `top` en üstteki elemanın bir üstündeki gözü gösteriyordur. Bu yüzden ilk iş `top`'u bir azaltmak, sonra oradaki değeri `x`'e almaktır.

Hoca burada bir tasarım seçeneğini açıkladı. `pop` fonksiyonundan iki türlü bir şey döndürülebilir: Yığının en üstündeki eleman doğrudan `return x` ile döndürülebilir ya da işlemin başarılı olup olmadığı döndürülebilir. Çıkan eleman da parametre üzerinden alınır. Hangisinin seçileceği tasarıma ve algoritmanın o anki ihtiyacına bağlıdır, ikisi de doğrudur. Burada örnek olması için ikincisi seçildi: `return 0` yığın boş olduğu için `pop` yapılamadı, `return 1` `pop` yapıldı demektir. Çağıran yer de buna göre işlem yapabilir.

Soru: `pop` en üsttekini çekmiyor muydu, neden `x` parametresine ihtiyaç var?

Cevap: Çünkü fonksiyon başarı durumunu döndürüyor. Bu durumda çıkan elemanı dışarı vermek için `x` gerekir. Fonksiyon `return x` ile tasarlansaydı buna gerek kalmazdı.

4.7 `peek` Fonksiyonu

Hoca yazılabilecek bir fonksiyon daha olduğunu söyledi ve onu öğrencilere bıraktı: `peek`.

Önce `pop`'un gerçekte ne yaptığını netleştirdi. Yığında yalnızca A olsun. `pop` deyince `x`'e A gelir ve `top` 0'ı gösterir. "Almak" diye bir şey aslında yoktur: A orada durmaya devam eder. Sonra `push C` denirse C, A'nın üzerine yazılır. Yani `pop` edilen eleman dizide silinmez, yeni gelen onu ezer.

`pop` etmek, "bu elemana artık ihtiyacım yok" demektir. Örneğin `F2`'yi çağırdığınızda yığın, `F2`'den nereye döneceğinizi bulmak ve `F2`'ye aktarılan parametreleri tutmak için kullanılır. `F2`'den döndüyseniz dönüş adresini bir daha saklamanın gereği yoktur, `pop` edilir ve iş biter.

Bazen ise elemanı çıkarmadan yalnızca en üstte ne olduğunu bilmek istersiniz. `peek` bunu yapar. Yığında A, B, C varsa ve `top` C'nin bir üstünü gösteriyorsa `peek` C'yi döndürür, ama `top`'u oynatmaz. Bundan sonra `push` denirse eleman C'nin üstüne yerleşir, `pop` denirse C çıkar.

4.8 Dinamik Bellek Sorusu

Soru: Yığını sabit dizi yerine dinamik bellekle yapsak, en üstteki elemanı `pop` edince diziden çıkarsak, o yer yığında yer kaplamasa ve başka programlar kullanabilse daha iyi olmaz mı?

Cevap: Olmaz. Yığın bölgesinin büyüklüğü ihtiyaca göre belirlenir. Hocanın örneğiyle yığın alanı bir megabayt olsun, o kadardır. Sürekli hareket olan bir yığında her eleman çıktığında `free`, geldiğinde `realloc` yapmak verimli bir çözüm değildir. Dizinizi dinamik olarak ayırabilirsiniz. Yığın boyutu dışarıdan okunabilir, duruma göre 20, duruma göre 20 bin olabilir. Ama bir kez ayırırsınız ve sonra hep o bölgeyi kullanırsınız.

Hoca aynı hatanın quizde de görüldüğünü söyledi: Yeni eleman geldikçe döngü içinde `realloc` yapanlar vardı. Kabul edilebilir olan, işin sonunda bir kez küçültmektir. Örneğin renk sorusunda en fazla $k/3$ renk olabileceği için o kadar yer tuttunuz, ama bütün resimde bir tek renk kullanılmış olabilir. O zaman iş bitince bir kez `realloc` yapmak doğrudur. Her yeni renk geldikçe `realloc` yapmak ise zaman açısından sıkıntılıdır. Çünkü arkada boş yer yoksa `realloc` elindekileri başka yere taşır, bu da vakit kaybıdır.

> **Hocanın vurgusu:** Bunları tasarımlarınızda düşünün. Korkmanıza gerek yok. Aklınızın bir köşesinde bulunsun, yavaş yavaş dikkat edersiniz.

5. İnfix İfadeyi Postfix İfadeye Çevirme

5.1 İnfix ve Postfix

`3 + 5 * 4 / 2 - 6` bir ara (infix) ifadedir. Normalde kullandığımız, işaretlerin sayıların arasında durduğu biçimdir. İşlem sırasına göre çevirdiğimizde son (postfix) ifade elde edilir. Hoca bunu Ayrık Matematik dersinden hatırlattı.

Bu ifadenin postfix karşılığı `3 5 4 * 2 / + 6 -` şeklindedir. Nedeni işlem sırasıdır:

- 1. işlem: 5×4
- 2. işlem: sonucu 2'ye böl
- 3. işlem: 3 ile topla
- 4. işlem: 6 çıkar

Bir öğrenci prefix gösteriminin de olduğunu hatırlattı. Hoca, prefix'te işaretlerin önce yazıldığını söyledi. Dağılmamak için onu ayrıca işlemedi, çünkü yığınla nasıl yapıldığını postfix üzerinden anlatacaktı.

Asıl sorun şudur: `3 + 5 * ...` ifadesinde toplamayı bir türlü yapamıyorum. Ondan öncelikli işlemler bitince yapabileceğim. O hâlde çarpma ve bölme yaparken toplamayı yapacağımı aklımda tutmam gerekir. Bu "aklımda tutma" işini yığın yapar.

5.2 İşleç Öncelikleri

Çevirme için işaretlerin tutulduğu bir işleç yığını (operator stack) kullanılır. İşlem öncelikleri `push` ve `pop` için ayrı tanımlanır (ekrandan, 01:31:00; kısa nottan):

Öncelik	Yığına koyarken (`push`)	Yığından çıkarırken (`pop`)
En yüksek	`(`	`* /`
Orta	`* /`	`+ -`
En düşük	`+ -`	`(`

Parantez içi öncelikle yapılır. Bu yüzden `(` yığına konurken en yüksek önceliklidir. Yığından çıkarken ise en düşük önceliklidir, çünkü kapa parantez gelmeden aç parantezi dışarı çıkaramazsınız. Parantez içindeki bütün işlemler bitmeden dışındakiler yapılamaz.

Hocanın örneği `3 + 5 * (2 + 4 - 7)` ifadesidir. Çarpı işareti olmasına rağmen, 5 ile çarpabilmek için önce `2 + 4 - 7` işleminin bitmiş olması gerekir.

Algoritmanın kuralları şöyledir:

- Sayı gelince doğrudan çıkışa yazılır.
- İşleç gelince yığının üstüne `peek` ile bakılır. Üstteki işlecin çıkma önceliği gelen işlecinkinden büyük veya eşitse `pop` edilip çıkışa yazılır. Bu, çıkamayan bir işleç kalana kadar sürer. Sonra gelen işleç yığına konur.
- `)` gelince `(`'a kadar bütün işleçler çıkarılıp çıkışa yazılır. `(` da çıkarılır ama yazılmaz.
- İfade bitince yığında kalan her şey sırayla çıkarılır.

5.3 Örnek: `3 + 5 * 4 / 2 - 6`

Hoca her adımda neden beklediğini açıkladı. `+` gelince toplamayı yapıp yapamayacağımı bilmiyorum, çünkü sonraki işaret çarpı ise yapamam, eksi olsaydı yapardım. Bunu ancak sonraki işarete gelince öğrenirim. `*` gelince `+` çıkamaz, ama çarpmayı da hemen yapamam: Belki sonraki `(` olacaktır. Bu yüzden o da bekler.

Gelen	İşlem	Yığın (alt → üst)	Çıkış
3	yaz		3
+	push	+	3
5	yaz	+	3 5
*	+ çıkamaz, push	+ *	3 5

Gelen	İşlem	Yığın (alt → üst)	Çıkış
4	yaz	+ *	3 5 4
/	* eşit öncelikli, çıkar; + çıkamaz; push	+ /	3 5 4 *
2	yaz	+ /	3 5 4 * 2
-	/ çıkar; + eşit öncelikli, çıkar; push	-	3 5 4 * 2 / +
6	yaz	-	3 5 4 * 2 / + 6
son	kalanları çıkar		3 5 4 * 2 / + 6 -

/ gelince yapılan işe dikkat edin: Yığının üstüne peek ile bakılıyor, * var. Öncelikleri aynı olduğu için * pop edilip çıkışa yazılıyor. Sonra yine peek yapılıyor, + var ve çıkamıyor. / da hemen yapılamaz, çünkü belki sonrasında yine bir aç parantez gelecektir.

Sonuç, başta doğrudan yazılan postfix ifadeyle aynıdır.

5.4 Örnek: `3 + 5 * (4 + 2 * 7) / 6`

Hoca ifadeyi kurarken ara sonuçları hesapladı: Parantez içi 18, $5 \times 18 = 90$ ve ardından 6'ya bölme.

Gelen	İşlem	Yığın (alt → üst)	Çıkış
3	yaz		3
+	push	+	3
5	yaz	+	3 5
*	+ çıkamaz, push	+ *	3 5
(	push	+ * (	3 5
4	yaz	+ * (	3 5 4
+	(çıkamaz, push	+ * (+	3 5 4
2	yaz	+ * (+	3 5 4 2
*	+ çıkamaz, push	+ * (+ *	3 5 4 2
7	yaz	+ * (+ *	3 5 4 2 7
)	*, + çıkar; (çıkar, yazılmaz	+ *	3 5 4 2 7 * +
/	* eşit öncelikli, çıkar; + çıkamaz; push	+ /	3 5 4 2 7 * + *
6	yaz	+ /	3 5 4 2 7 * + * 6
son	kalanları çıkar		3 5 4 2 7 * + * 6 / +

İçerideki + geldiğinde (çıkamaz, çünkü çıkma önceliği en düşüktür. Dışarıdaki * da çıkamaz, çünkü üzerinde aç parantez vardır. Kapa parantez, parantez içindeki işlem grubunun bittiğini gösterir.

> **Hocanın vurgusu:** Kapa parantez gelince aç paranteze kadar olanların tamamı dışarı çıkarılır ve postfix ifadeye eklenir. Aç parantez de çıkarılır, ama bir işlem işareti olmadığı için eklenmez. Aç ve kapa parantez yalnızca bir işlem grubunu kontrol altında tutmayı sağlar.

Sonuçta işlem sırası da gözle görülür: önce 2×7, sonra 4 ile toplama, sonra 5 ile çarpma, sonra 6'ya bölme, en son 3 ile toplama. Hoca bu sırayı ekranda işaretlerken bir çizgiyi geri aldı ve "gördüğünüz gibi stack kullanarak geri alıyoruz" dedi.

5.5 Soru: Sayı Gelince İşleç Çıkarılamaz mı?

Soru: İlk örnekte çarpıdan sonra 4'ü gördüğümüzde bölüyü görmemize gerek var mıydı? Parantez gelecekse 4'ün önüne gelmesi gerekirdi. Parantezi görmediğimize göre çarpıyı hemen yazamaz mıydık?

Cevap: Bu senaryo için söylediğin doğru, çünkü burada dört işlem dışında öncelikli olabilecek tek şey aç parantezdir. Ama algoritma tasarımsal olarak standart çalışır: Bir işaret geldi, bir sonraki işaret gelene kadar yığında bekler. Öbür türlü çarpma ve bölmeye özel bir iş yapmış olursun. Üs alma gibi yeni işaretler gelirse algoritmayı her duruma göre değiştirmen gerekir. Bir de şu sorun var: Çarpıyı çıkardın, toplamayı çıkarabilecek misin? Bilmiyorsun, görmek için yine sonraki işareti beklemek zorundasın. Bölü değil eksi olsaydı artı da çıkacaktı. Algoritma "şu durumda, bu durumda" kontrolleriyle dolar. Onun yerine sakin sakin beklersin. Bölü geldiyse "çarpı çıkar, artı çıkamaz" kararını bir seferde verirsin.

6. Postfix İfadeyi Yığınla Hesaplama

Hoca önce gerçek bir ifadenin nasıl göründüğünü söyledi. Gerçekte `A + B * C / D - E` gibi değişkenli ifadeler yazılır. Derleyici değişkenlerin sayı karşılıklarını bellekten bulup iş yapar. Hoca, değişkenin değeri neydi diye kafamız karışmasın diye örnekleri doğrudan sayılarla gösterdi.

Bir postfix ifadeyi hesaplamak için bu sefer değerler için bir yığın kullanılır. Kural kısadır:

- Sayı gelince yığına yazılır.
- İşlem işareti gelince yığının en üstündeki iki değer çekilir, aralarında işlem yapılır, sonuç yığına geri yazılır.
- İfade bitince yığında kalan değer `pop` edilir. Sonuç odur.

İşlem iki değer arasında yapıldığı için işaret her seferinde iki değer çeker. Örnek, 5.4'te elde edilen `3 5 4 2 7 * + * 6 / +` ifadesidir (ekrandan, 01:41:40; kısa nottan):

Gelen	İşlem	Yığın (alt → üst)
`3 5 4 2 7`	sırayla yığına yaz	3 5 4 2 7
`*`	$2 \times 7 = 14$	3 5 4 14
`+`	$4 + 14 = 18$	3 5 18
`*`	$5 \times 18 = 90$	3 90
`6`	yığına yaz	3 90 6
`/`	$90/6 = 15$	3 15
`+`	$3 + 15 = 18$	18

Sonuç 18'dir. Sıraya dikkat edilmelidir: Hoca bölmede alttaki değeri üsttekine böldü, yani önce çekilen değer sağ işlenendir. $90/6$ yapılır, $6/90$ değil. İşlemler bitince yığındaki değer `pop` edilip örneğin `x`'e atanıyorsa bellekte `x`'in olduğu yere 18 yazılır.

Soru: Atamada da böyle bir mekanizma mı kullanılıyor? Atama işareti aritmetik işlemlerden düşük öncelikli olarak mı işleniyor?

Cevap: Hayır. Atama görünce atama yapacağını anlıyor. Burada bir sol işlenen (left operand), bir de sağ işlenen (right operand) vardır. `=` işaretine postfix'e çevrilecek bir işleç muamelesi yapmaz. Bellekte `x`'in yerine gider ve sağ tarafta yapılan işlemin sonucunu oraya yazar.

7. Derleyicinin İki Yığınla Hesaplaması

7.1 Aradan Gelen Soru: `peek` Ne Döndürür?

Aradan sonra bir öğrenci `peek` ile ilgili bir soru sordu.

Soru: `peek`, `empty` ve `full` gibi doğru/yanlış mı döndürecek, yoksa değeri mi?

Cevap: Değeri döndürecek. Yığında A, B, C varsa ve `top` C'nin üstünü gösteriyorsa, `peek` "C var" demelidir. Örneğin `x = peek(...)` denir ve `x`'e C gelir.

Soru: Daha karmaşık bir eleman olsaydı `peek` pek işe yaramazdı herhalde?

Cevap: Hoca bunun için `peek`'in neden kullanıldığını hatırlattı. Dönüşümde çarpıyı çıkarıp çıkarmamaya karar vermek için yığının üstüne `peek` ile bakılıyor. İşine gelirse çıkarıyorsun, gelmezse çıkarmıyorsun. `peek`'te `top` oynatılmaz, yalnızca en üstteki gösterilir. `pop` dersen ise çarpı çıkmış olur ve artık orada yoktur.

7.2 Mantık

Hoca bu kez derleyicinin bir aritmetik ifadeyi nasıl çözdüğünü gösterdi. İşleç yığınındaki çıkarma kararları ile değer yığınındaki hesaplama burada birlikte yürür. İki yığın kullanılır:

- İşlenen yığını (operand stack): değerler için.
- İşleç yığını (operator stack): işaretler için.

Bir işleç yığından çıkınca işlenen yığınının en üstündeki iki değer çekilir, işlem yapılır, sonuç işlenen yığınına geri yazılır. Hoca önce daha karmaşık bir ifadeyle başlayıp vazgeçti ve mantık anlaşılsın diye basit bir örneğe geçti.

> **Ek bilgi (derste anlatılmadı):** Burada anlatılan, işlem önceliğinin mantığını gösteren bir modeldir. Değişkenlerin değeri derleme anında bilinmediği için derleyici `a + b * c` gibi bir ifadeyi derlerken hesaplamaz, onu hesaplayan makine kodu üretir. `gcc -S` ile denendi: Bu ifade için çarpma (`imull`) ve toplama (`addl`) komutları üretildi. Yalnız sabitlerden oluşan `5 + 4 * 2 - 1` ise doğrudan 12 olarak koda yazıldı.

7.3 Örnek: `5 + 4 * 2 - 1`

Gelen	İşlenen yığını	İşleç yığını	Açıklama
5	5		
+	5	+	sonraki işaret bilinmiyor
4	5 4	+	
*	5 4	+ *	* daha öncelikli, + çıkamaz
2	5 4 2	+ *	
-	13	-	* çıkar: $4 \times 2 = 8$. + çıkar: $5 + 8 = 13$
1	13 1	-	
son	12		- çıkar: $13 - 1 = 12$

İşaret kalmayınca 12 yığından çıkarılır. `x = ...` denmişse derleyicinin yaptığı, bu sonucu bellekte `x` değişkeninin bulunduğu yere yerleştirmektir.

7.4 Örnek: `3 * (5 + 4 / 1) - 2`

Gelen	İşlenen yığını	İşleç yığını	Açıklama
3	3		
*	3	*	
(	3	* (	
5	3 5	* (	
+	3 5	* (+	(çıkamaz
4	3 5 4	* (+	
/	3 5 4	* (+ /	/ öncelikli, + çıkamaz
1	3 5 4 1	* (+ /	
)	3 9	*	/ çıkar: $4/1 = 4$. + çıkar: $5 + 4 = 9$. (çıkar, işlem yapılmaz
-	27	-	* çıkar: $3 \times 9 = 27$
2	27 2	-	
son	25		- çıkar: $27 - 2 = 25$

Hoca değerlerin çekiliş sırasını da belirtti: + çıkınca önce 4, sonra 5 çekilir ve $5 + 4$ yapılır. Aç parantez üzerinde hiçbir işlem yapılmaz. O yalnızca, kapa parantez gelince aç paranteze kadar olan işaretleri çıkarmaya yarar.

7.5 Örnek: `3 * (5 + 4 * 2 - 1)`

Hoca bu örneği bir noktayı görmemiz için ekledi: Parantezin içindeki işlemler de kendi önceliklerine göre çıkar.

Gelen	İşlenen yığını	İşleç yığını	Açıklama
`3`	3		
`*`	3	*	
`(`	3	* (	
`5`	3 5	* (	
`+`	3 5	* (+	
`4`	3 5 4	* (+	
`*`	3 5 4	* (+ *	+ çıkamaz
`2`	3 5 4 2	* (+ *	
`-`	3 13	* (-	* çıkar: $4 \times 2 = 8$. + eşit öncelikli, çıkar: $5 + 8 = 13$. (çıkamaz
`1`	3 13 1	* (-	
`)`	3 12	*	- çıkar: $13 - 1 = 12$. (çıkar
son	36		* çıkar: $3 \times 12 = 36$

Yani kapa parantez gelmeden de parantez içinden işlem çıkabilir. Yalnızca aç parantez, kapa parantez gelene kadar yerinde kalır.

Sonuçla ne yapılacağı ifadenin kullanıldığı yere bağlıdır. `x = ...` ise 36, x'in durduğu yere yazılır. `return a * b` ise hesaplanan değer geri döndürülür. Bir fonksiyon çağrısının parametresinde `a - b` yazılmışsa hesaplanıp fonksiyona gönderilir. Bundan sonrası, sizin yapmak istediğiniz işle ilgilidir.

7.6 Öncelik Tablosu ve Fonksiyon Çağrıları

Soru: Çarpmanın toplamaya göre önceliği gibi sıralamayı bir yerde tutacak mıyız?

Cevap: Evet, tutman gerekir. Bir işlem önceliği tablosu olmalıdır. Örneğin `+` ve `-` için 1, `*` ve `/` için 2. Bunlar bir struct'ta tutulabilir. `+` üzerine `*` gelince "2, 1'den büyük, o hâlde `+` çıkamaz" denir. Başka türlü yapılamaz. Örnekler kolay olsun diye dört işlemle verildi. Ama yalnızca C'de bile kaydırma (shift), `++`, `--` gibi pek çok işaret vardır. Hepsinin önceliği tutulur. Hoca gerçekte derleyicinin de böyle yaptığını söyledi.

Soru: Bu önceliklerde fonksiyon çağrıları da var mı?

Cevap: Burada anlatılan yalnızca işlem işaretleriyle ilgilidir. Derleyicinin çalışma mekanizmasının bütünü ayrı bir konudur, oraya girersek derleyici yazmamız gerekir. Burada yalnızca işlemlerin önceliklerine göre nasıl değerlendirildiğine bakıyoruz.

8. Parantez Denetimi

Derleyicilerin yığını kullandığı bir yer de parantez denetimidir. Excel'de de bir formül yazarken parantez hatası yaparsanız düzeltir. C kodu yazdığınızda da derleyici parantezlerin eşit sayıda ve doğru sırada olup olmadığına bakar.

Sadece aç ve kapa parantezleri saymak yetmez. Hocanın gösterdiği bir ifadede iki aç, iki kapa parantez vardı ama önce kapama, sonra açma geliyordu. Sayıları eşit, sırası yanlıştır.

> **Hocanın vurgusu:** Parantezlerin yalnızca eşit sayıda olması yetmez. Sıralarının da doğru olması gerekir.

Yığınla çözüm çok basittir:

- `(` gelince `push` edilir.
- `)` gelince `pop` edilir.

İşlem bittiğinde yığının boş olması, `pop` ederken de yığında eleman bulunması beklenir. Hoca üç durum gösterdi:

Durum	İfade	Olan	Sonuç
Doğru	`(B * C) - (D * E)`	`(` push, `)` pop, `(` push, `)` pop. Sonda yığın boş	Hata yok
Sıra yanlış	`) B * C (`	İlk `)` gelince boş yığından `pop` denenir	“Parantez sırası yanlış”
Fazla aç parantez	iki `(` var, `)` yok	İki `(` yığına yazılır. İşlem bitince yığın boş değil	“Aç parantez fazlası var”

Hata mesajı daha da yararlı hâle getirilebilir. İfade bittiğinde yığın boş değilse yığın boşalana kadar `pop` edilir ve “üç tane parantez fazla” gibi bir mesaj verilebilir. İki kapa parantez fazlaysa, her birinde `pop` boş yığınla karşılaşır ve “iki kapa parantez fazla” denebilir.

> **Ders sonrası not:** Altyazıda 02:20:18’de “stackten push işlemi yaparsınız empty olana kadar”, 02:20:56’da “pop geldiğinde” geçiyor. Kastedilen, yığın boşalana kadar `pop` yapmak ve kapa parantez gelince `pop` etmektir. Hoca kuralı önceki cümlelerde doğru vermiştir.

Hoca yığın konusunu burada toparladı: Son derece eğlenceli bir yapı olduğunu, deyip geçmemek gerektiğini söyledi. Bundan sonra geri tuşuna basarken ya da fonksiyon çağırırken “burada yığın kullanılıyor” diyebiliriz. Hangi işlerde sistemin yığın kullandığını kendimiz de düşünebiliriz. Buradaki hedef, son gelene ilk hizmetin verilmesidir. Araya dalıp beşinci gözdeki elemanı kullanmak istenmez. Hem istenmediği için hem de yapılamasın diye yığın tanımlanır. Bir hiyerarşi vardır ve ona uygun veri yapısı yığındır.

> **Hocanın vurgusu:** Denemeden bunların hiçbirini hazmedemezsiniz. Yığın işlemleriyle kendiniz oynayın, nasıl yapıldığını görün.

9. Geri Alma ve Yineleme: İki Yığın

Soru: Geri alırken yığın kullanıyoruz. Peki sildiğimiz bir şeyi sonradan geri yüklemek için ne yapmalıyız? Yığının tersini mi kullanacağız?

Cevap: Güzel soru. Bu her problemde gerekmez. Örneğin parantez denetiminde böyle bir ihtiyaç yoktur. Ama ileri (forward) ve geri (backward) ikisi de gerekiyorsa iki yığın kullanılır. Çıkarılanlar da bir yerde tutulur ki istendiğinde geri verilebilsin.

Hocanın gösterdiği iz şöyledir. Önce birinci şey, sonra ikinci şey yazıldı:

İşlem	İlk yığın (alt → üst)	İkinci yığın (alt → üst)	Ekranda
Birinci ve ikinci yazıldı	birinci, ikinci		ikisi de var
Geri	birinci	ikinci	birinci
Geri		ikinci, birinci	hiçbiri
İleri	birinci	ikinci	birinci

İleri denince ilk olarak en son geri alınan verilir. Hoca ekranda kullandığı programı da denedi: O programda yalnızca son geri alınan geri verilebiliyordu. Hepsinin defalarca kurtarılması isteniyorsa iki yığın kullanılır. Hoca pek çok metin düzenleyicinin bu iş için iki yığın kullandığını söyledi.

10. Kuyruk (Queue)

10.1 İlk Giren İlk Çıkar

Hoca kuyruğu, yığının “kardeşi” olarak tanıttı. Mantık farklıdır, ama amaç yine aynıdır: Bilgilere belli bir hiyerarşiye göre erişilsin, aradan istenen veriye erişilmesin. Yine dizi kullanılır,

farklı bir yapı yoktur.

Kuyruk, günlük hayattan iyi bildiğimiz gibi ilk giren ilk çıkar (FIFO, first in first out) yapısıdır. Kuyruğun önünde kim bekliyorsa hizmet ona verilir. Kuyruğun başına `front`, sonuna `rear` denir. Hizmet baştakine verilir, yeni gelen sona yerleşir.

Kullanım yerleri:

- **Ağdaki ortak yazıcı:** Herkes çıktısını oraya gönderir. Kimin çıktısı önce geldiyse o basılır, diğerleri bekletilir.
- **Giriş/çıkış tamponları (I/O buffer):** Bilgiler tamponda bekler, sırayla işlenir. Dosya giriş/çıkışında da kullanılır.
- **İşlemci zamanlaması (CPU scheduling) ve disk zamanlaması (disk scheduling):** Kaynaklar birden fazla kullanıcı tarafından kullanılıyorsa kuyruk vardır.
- **İşletim sistemleri ve ağ:** Çok kullanılan bir veri yapısıdır.
- **Banka kuyruğu uygulaması:** Yeni gelen numarasını alınca kuyruğa eklenir. Ekranda örneğin 79 yanıyorsa, elindeki kağıtta 79 yazan kişiye hizmet verilir. O bitince sıra bir sonrakine geçer.

10.2 Normal Dizide Kaydırmak Neden Verimsiz?

Önce kuyruğu normal bir dizi üzerinde düşünelim. 0, 1, 2, 3, 4. gözlerde sırasıyla A, B, C, D, E olsun. A'nın işi bitince B'nin başa gelmesi gerekir. Bu durumda herkesi bir öne kaydırırsınız.

Eleman eklemek sorun değildir, sona eklenir. Ama her biten işte herkes kaydırılır. Kaydırmak, elemanları bir önceki adrese taşımak demektir. Çok elemanlı bir kuyrukta bu, her çıkışta n'lik bir iştir (ara adım eklendi: tam sayı, n elemanlı kuyrukta $n-1$ taşımadır; bu örnekte 4). Bu verimli değildir.

Soru: Yığındaki `top` gibi bir değişken tutsak olmaz mı?

Cevap: Evet, söylediğin dairesel kuyruk (circular queue) mantığıdır.

10.3 Dairesel Kuyruk

Elemanlar kaydırılmaz. Dizi dairesel düşünülür. Yalnızca `front` ve `rear` hareket ettirilir. A'nın işi bitince A silinmez bile. `front` B'yi göstermeye başlar. Böylece A'nın durduğu göz bizim için boş sayılır. `rear`, yeni gelenin yazılacağı gözü gösterir. Yeni eleman oraya yazılır ve `rear` bir ilerler. Dizinin sonuna gelince başa döner.

Hoca bunu önce A, B, C, D, E ve bir boş gözle anlattı: A bitince `front` B'ye geçer. F gelince boş göze yazılır ve `rear`, A'nın durduğu gözü göstermeye başlar. Sonra daha ayrıntılı bir örnek yaptı (ekrandan, 02:43:15; kısa nottan). 5 gözlü bir dizi:

Durum	0	1	2	3	4	`front`	`rear`
Başlangıç	A	B	C	D		0	4
A bitti	(A)	B	C	D		1	4
E geldi	(A)	B	C	D	E	1	0
F geldi	F	B	C	D	E	1	1

Parantezli A, gözde hâlâ duran ama artık kuyruğa ait olmayan değerdir. F yerleşince `front` ve `rear` aynı gözü gösterir. Hoca bunu "kuyruğunuz dolu" diye yorumladı. G gelirse "full" denir ve daha fazla eleman yerleştirilemez.

> **Ders sonrası not:** Hoca `front` ile `rear`'ın aynı gözü göstermesini "dolu" diye açıkladı. Bu gösterimde `rear` bir sonraki boş gözü gösterdiği için kuyruk boşken de `front == rear` olur. Yukarıdaki son durumdan beş eleman çıkarılınca, python ile yapılan izde `front` ve `rear` yine 1 oldu, ama kuyruk boştu. Yani bu eşitlik tek başına dolu ile boşu ayırmaz. İki çözüm ayrı ayrı python ile denendi: Eleman sayısını bir sayaçta tutmak ya da bir gözü hep boş bırakıp `(rear + 1) % MAX == front` durumuna "dolu" demek. Kendi `isFull` ve `isEmpty` fonksiyonlarınızı yazarken bunu hesaba katın.

Hoca aynı mantığı, kuyruk ilerledikten sonraki bir anla pekiştirdi: A ve B bitmiş, `front` C'dedir. Arkadan X, Y, Z gelmiştir ve `rear` artık B'nin durduğu gözü gösterir. W gelirse B'nin üzerine yazılır. Kural kısadır: `rear` neresiyse yeni gelen oraya, `front` neresiyse hizmet ona. Baştakinin işi bitince `front` bir sonrakine geçer.

Soru: Dairesel kuyruk da yığın gibi sabit uzunlukta mı?

Cevap: Evet, bu hâli sabit uzunluktadır. Kuyruk en fazla belli sayıda eleman alır ve `front` ile `rear`'ın nerede olduğu bilinir. Dizi dinamik ya da statik olabilir, ama elemanlar ardışık durur ve en büyük eleman sayısı bellidir.

10.4 Kuyruk İşlemleri

İki temel işlem vardır:

- `enqueue`: Kuyruğa eleman yerleştirir. Eleman kuyruğun en sonuna gider.
- `dequeue`: Kuyruğun başındaki elemanı çıkarır. Market kuyruğunda en öndeki kişi alışverişini tamamlayıp gider. Yazıcıda çıktım basılınca arkamdaki kişininki basılır. Tamponda sıradaki işlem yapılır.

Yanlış işlemleri engellemek için `isFull` ve `isEmpty` ile kontrol şarttır. Örneğin kuyruk boşken yazdırmaya çalışmak ya da kuyruk doluyken yeni eleman yerleştirmeye çalışmak engellenmelidir.

Üçüncü olarak `peek` vardır. Kuyruğun başında kim olduğunu gösterir, kuyrukta değişiklik yapmaz.

10.5 Bağlı Liste ile Kuyruk

İkinci yol bağlı liste kullanmaktır. Mantığı daha kolaydır. `head` kuyruğun başını, `last` sonunu gösterir. Hocanın örneği:

Olay	Kuyruk	`head`	`last`
3 geldi	3	3	3
5 geldi	3 → 5	3	5
7 geldi	3 → 5 → 7	3	7
3'ün işi bitti	5 → 7	5	7

Yeni gelen, kuyruğun sonunun gösterdiği adresin arkasına eklenir. İşi biten de `head`'in gösterdiği adresten alınır. Kaydırma gerekmediği için bu yorucu bir yapı değildir. Çok fazla hareket varsa kullanılabilir.

İkisinin karşılaştırması şöyledir:

- **Dairesel kuyruk:** Ekstra gösterici tutulmaz. Yalnızca `front` ve `rear` oynatılarak işlem yapılır. Bu yüzden güzel bir yapıdır. En büyük eleman sayısı sabittir.
- **Bağlı liste:** Kuyruğun ne kadar büyüyüp küçüleceği bilinmiyorsa ya da sınır konmak istenmiyorsa kullanılabilir.

  Hocanın vurgusu: `enqueue`, `dequeue`, `isFull`, `isEmpty` fonksiyonlarını dairesel kuyruk mantığıyla kendi başınıza yazmaya çalışın. İnternette bu kodları çok çabuk bulabilirsiniz, ama bakmayın. Kendiniz uğraşırsanız öğrendikleriniz daha kalıcı olur.

Dersin Toparlaması

Bu derste son giren ilk çıkar mantığıyla çalışan yığın ve ilk giren ilk çıkar mantığıyla çalışan kuyruk görüldü. İkisi de aslında sıradan bir dizi kullanır. Fark, diziye yalnızca belli kurallarla erişilmesidir. Yığın için `push`, `pop`, `peek`, `empty` ve `full` yazıldı. Boş ve dolu kontrollerinin, adres göndermenin ve `else` yazmanın neden önemli olduğu tartışıldı.

Yığının en önemli uygulaması aritmetik ifadelerdir: İnfix ifadenin işleç yığınıyla postfix'e çevrilmesi, postfix ifadenin değer yığınıyla hesaplanması ve iki yığının birlikte kullanılmasıyla

ifadenin okunurken hesaplanması. Bunlara parantez denetimi ile iki yığınla geri alma ve yineleme eklendi. Kuyrukta ise elemanları kaydırmanın verimsizliğinden yola çıkılarak dairesel kuyruğa, sonra da bağlı liste ile kuyruğa ulaşıldı. Dairesel kuyruk fonksiyonlarını yazmak öğrencilere bırakıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.